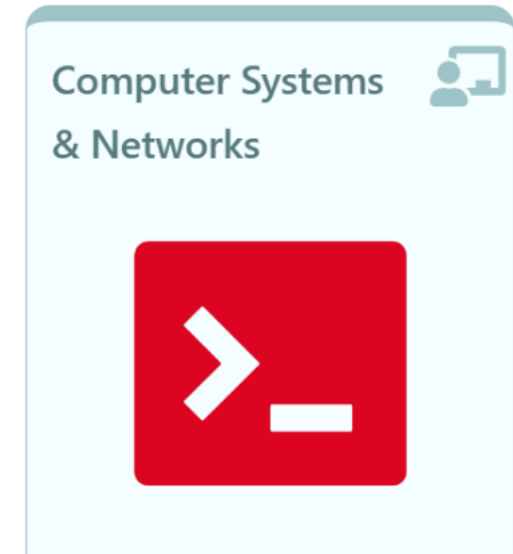


Computer Systems & Networks



sed Solns

Fundamentals

print and printf

Loops

Extract and filter

5% Homework

sed Solns

pattern matching · column processing · 5% CA

Objectives

It's all about the **awk** processing language today.

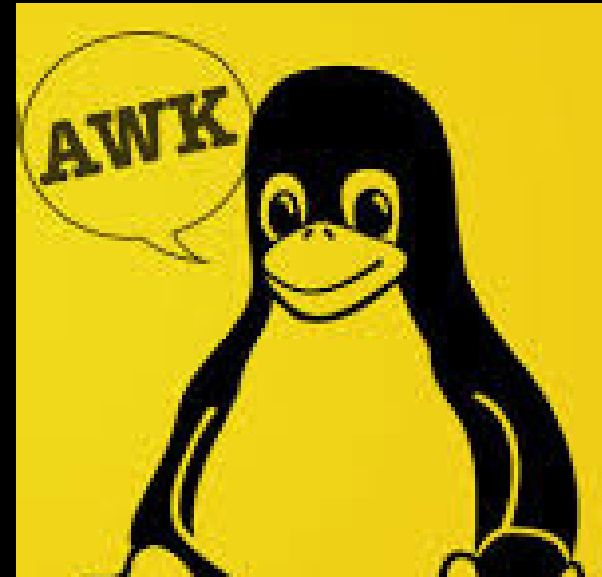
On successful completion, you should have understanding of **awk**

- to perform text pattern scanning and processing language
- as a line processor
- manipulating files that contain columns of data
- filtering lines returned by **grep**

SOLUTIONS FROM **sed** lab

- Check if an old temporary file exists called **employee.bak** and if so, then delete this
-

\$awk 'pattern { action }' file



\$awk 'pattern { action }' file

- excellent tool for processing files with rows and column
- originally designed for pattern searching and text manipulation
- Basic function:
 - search files for lines that contain certain **patterns**
 - on match, perform some **action(s)**



Use straight
ASCII quotes,
not curly
quotes!

\$awk 'pattern { action }' file

Manipulate and extract individual fields

- **awk** can process text files.
- assigns variables for each data field found

\$0 for the whole line.

\$1 for the first field.

\$2 for the second field.

\$n for the nth field.

You can choose to:

- a. Print out only the first field of the **employee.txt** file
- b. Print out only the first and third fields of the **employee.txt** file

Common uses of **awk**

awk can:

- Break lines into fields (default delimiter is whitespace)
- use conditions to print only lines/columns you want
- format the output i.e. an empty line between each line

```
$cat employee.txt | awk '{print $0 "\n"}'
```

- format with **printf**

Manipulate and extract individual fields

awk

- divides a file into lines called *records*
- splits each record into *fields*

The *fields* are accessed by special variables:

- the first field is \$1, the second \$2, and so on.
- the special variable \$0 contains the entire record before splitting.

```
$awk '{ print $3, $1 }' employee.txt
```

Pattern matching

To use **awk** to extract the second field of each line:

```
awk '{print $2}' employee.txt
```

You can also add a pattern.

```
awk '$3=="manager" {print $2}' employee.txt
```

second field will only be printed if the first field is equal to *manager*

```
compsys@compsys-virtualbox:~$ cat employ*  
300 frank manager sales €5000  
400 caroline developer technology €5500  
100 mary sysadmin technology €7000  
200 rosanne manager marketing €9500  
500 eamonn dba technology €6000  
600 colm sysadmin technology €4500
```

Example:

```
compsys@compsys-virtualbox:~$ awk '/c/{print}' employee.txt
200 caroline developer technology €5500
300 mary sysadmin technology €7000
500 eamonn dba technology €6000
compsys@compsys-virtualbox:~$ awk '/C/{print}' employee.txt
compsys@compsys-virtualbox:~$
```

\$ awk 'pattern { action }'

- **awk** is a line-oriented language
- By default **awk** prints every line of data from the specified file
- **awk** treats tab or whitespace for file separator by default.

\$ awk '{print}' employee.txt

\$ awk '{print \$2}' employee.txt

\$ awk /C/ '{print \$2}' employ*

Pattern Match fields

- Provide a regular expression enclosed by slash **/** characters to match records:

```
$ awk '/caroline/ {  
    print $1, $2  
}' employee.txt
```

```
$ awk 'BEGIN { IGNORECASE = 1 }  
/caroline/ {  
    print $1, $2  
}' employee.txt
```

```
$ awk '$1==200 {  
    print $1, $2  
}' employee.txt
```

can also use fields and a comparison expression as pattern matching

Should it be case insensitive?

awk Comparison & Logical Operators

awk has all the usual:

1) comparison operators

and 2) arithmetic operators

< less than

<= less than or equal to

== equal to

>= greater than or equal to

!= not equal to

> greater than

+ add

- subtract

* mult

/ divide

% modulo

<space> concatenate

Uses of awk: Filtering Lines (like grep)

```
$grep "Kilkenny" employee.txt
```

This finds and prints any line that contains the word "Kilkenny" — anywhere in the line.

Always use **straight double quotes** ("Kilkenny"), not curly quotes.

- Can act like **grep** but with more control.

```
$awk '$3 == "Kilkenny"' employee.txt
```

Uses of awk: Text Transformation

Print the third and first field of employee.txt

```
$awk '{  
    print $3, $1  
}' employee.txt
```

Uses of awk: Text Parsing / Field Extraction

Breaks lines into fields (default delimiter is whitespace).

```
$echo "123 Caroline Kilkenny" | awk '{ print $3 }'
```

Uses of awk: Case insensitive Pattern Matching

Process only lines that match specific conditions

- The slashes **/.../** in awk are used to **define a pattern to match**.

```
awk '  
/error/ {  
    print $0  
}  
' myerrors.txt
```

Look through myerrors.txt, and print every line that contains the word "error".

Note: Case Sensitivity

The pattern **/error/** is **case-sensitive** so you mightn't want this:

```
$awk 'BEGIN {  
    IGNORECASE = 1  
}  
/error/ {  
    print $0  
}' myerrors.txt
```

Uses of awk: Data Processing / Reports

\$awk 'pattern { action }' file

- Perform calculations or format output

```
awk '{  
    sum += $2  
}  
END {  
    print "Total:", sum  
}' expenses.txt
```

Common Uses of *awk* in Bash

Field Processing

ID	Name	Salary
---	---	---
---	---	---
---	---	---

Extract and manipulate columns.

Reports & Summaries



Generate and format reports.

Text Filtering



Match and filter lines of text.

Data Calculations



Calculate sums and averages.

Log Analysis

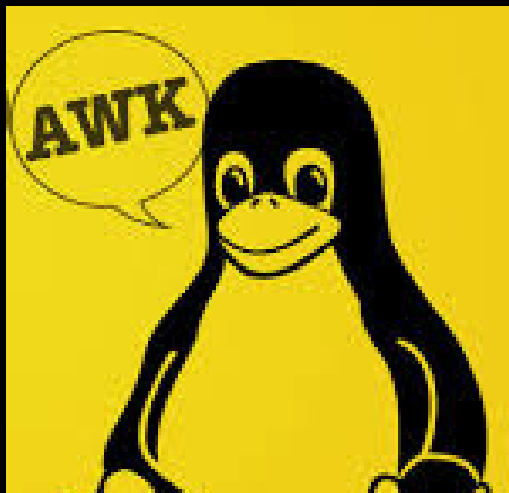


Analyze and parse log files.

Pattern Matching



Search for specific patterns.



for column processing

awk
variables

awk How to change the field separator

- Two actions:

```
awk 'BEGIN {  
    FS = ":";  
    OFS = "-";  
}  
{  
    print $2, $3  
}' employee.txt
```

FS stands for **Input
Field Separator**

OFS stands for **Output
Field Separator**

```
$cat employee.txt  
300:caroline.cahill:sales  
400:frank.walsh:manager
```

```
OUTPUT:  
caroline.cahill-sales  
frank.walsh-manager
```

1. Find the colons
2. Use **:** as the input separator but print the dash **-** as the output field separator

awk How to change the field separator

- Using **-F** on the command line

```
$ awk -F',' '{ print $2 }' employee.txt
```

Assuming **employee.txt** is a comma-separated file;

-F is easy for quick one-liners

- Setting **FS** inside the awk script

```
$awk 'BEGIN {  
    FS = "," #Use OFS = "," inside BEGIN if you want to control output formatting too  
}  
{  
    print $2  
}' employee.txt
```

BEGIN { FS = "," } sets the delimiter before reading any lines.

Using FS is better for longer or more readable awk scripts.

Manipulate and extract individual fields

- Built-in variables have values already defined in **awk**

<u>Variable name</u>	<u>Description</u>
FS	field separator (default space)
NF	No. of Fields; prints last field in the current record
NR	current record/line number=total count of processed records
\$0	whole line
\$n	n-th field
FNR	current record number in the current file

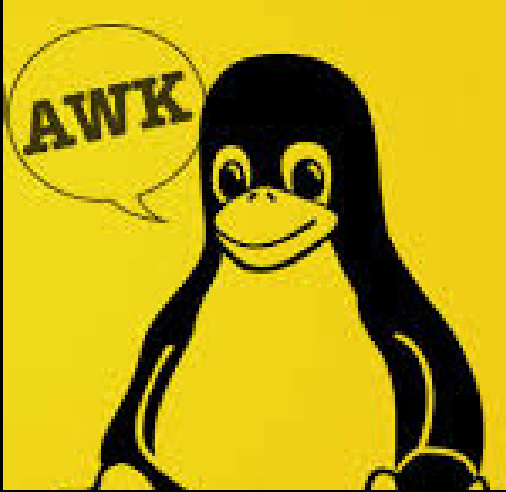
Storing contents of a specific field in a variable

```
emailaddr=$(grep -i caroline employee.txt | awk '{print $5}')
```

This assumes the email is the fifth field on each line i.e. **awk '{print \$5}'**

EXERCISE: what would be the output of:

```
awk '{print $5}' employee.txt | sort | uniq
```



for pattern matching

awk
printing

Printing specific output

```
compsys@compsys-virtualbox:~/temp/tutorial$ ls -l
total 8
drwxrwxr-x 4 compsys compsys 4096 Aug 31 13:49 Dec
drwxrwxr-x 2 compsys compsys 4096 Aug 31 13:02 Nov
```

EXERCISE: We want to extract only the owner, size, and file name:

ls -l | awk '{print \$3 \$5 \$9}'

poor output

ls -l | awk '{print \$9 " " \$5 " "\$3}'

#add spaces manually

ls -l | awk '{print \$5"\t"\$1}'

force a tab \t

ls -l | awk -v OFS='\t' 'NR>1 {print \$9, \$5, \$3}' #better again!

Printing user friendly specific output

```
compsys@compsys-virtualbox:~$ awk '/manager/ {print "The " $4 " manager name is: " $2 " who claimed expenses of: " $(NF)}' employee.txt
The sales manager name is: frank who claimed expenses of: €5000
The marketing manager name is: rosanne who claimed expenses of: €9500
compsys@compsys-virtualbox:~$
```

```
compsys@compsys-virtualbox:~$ awk '/[Mm]anager/ {print "The manager " $2 " claimed expenses of " $(NF)}' employee.txt
The manager frank claimed expenses of €5000
The manager rosanne claimed expenses of €9500
The manager Caroline claimed expenses of €1234
compsys@compsys-virtualbox:~$ cat empl*
300 frank manager sales €5000
400 caroline developer technology €5500
999 catherine dev tech €4500
100 mary sysadmin technology €7000
200 rosanne Manager marketing €9500
500 eamonn dba technology €6000
600 colm sysadmin technology €4500
echo "There are no $ currency, only € in employee.txt"
just another example katherine
666 Katherine dev tech €6000
123 Caroline Manager Sales €1234
```

Print fields using **printf**

- Use **print** when you want quick, simple output:

```
~$ awk '/caroline/ {print $1,$2}' employee.txt
```

- Use **printf** when you want **precise formatting** (aligned columns, fixed widths, decimals, etc).

```
compsys@compsys-virtualbox:~$ awk '{printf("%-40s %s\n", $1,$2)}' employee.txt
300                                frank
400                                caroline
999                                catherine
100                                mary
200                                rosanne
500                                eamonn
600                                colm
```

Modifiers for **printf** Formats

The **printf** command in **awk** allows you to print formatted output using format specifiers

<u>SYNTAX:</u>	<u>%[modifier]control-letter</u>
c	Prints numeric output as a string.
d	Prints an integer value.
f	Prints float values.
o	Prints an octal value.
s	Prints a text string.
e	Prints scientific numbers.

Modifiers for **printf** Formats

```
$printf "%4s", "computersystems"
```

- Inserting any number after the '%' sign forces the field to be expanded to this width (*at least*)
 - default is to pad with spaces on the left,
 - “-” causes padding on the right

```
$awk '{printf "%-10s %s\n", $1, $2}' employee.txt
```

Extract multiple fields from record

Print column one and three:

```
$ awk '{ print $1, $3 }' employee.txt
```

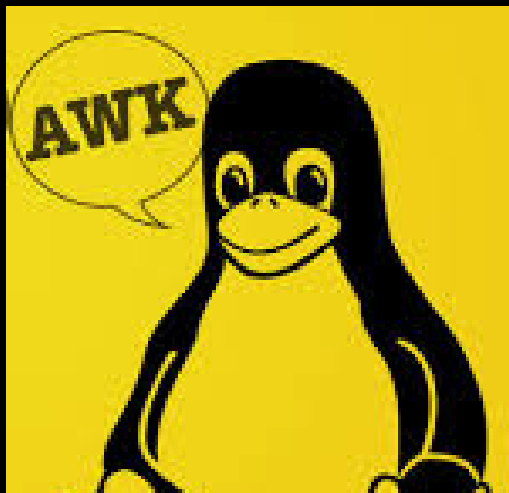
- You could use the **printf** function and provide a 40 character padding to the right of first column:

```
$ awk '{ printf("%-40s %s\n", $1, $3) }' employee.txt
```

-40s Format string: left-align (-) the first field as a string in a **40-character wide** column

%s Format the third field as a string, placed right after the first column

\n adds a newline at the end



syntax

BEGIN, ~~////~~
main and
END

BEGIN and END

- A **BEGIN** rule is executed once only, *before* the first input record is read.
- Likewise, an **END** rule is executed *once* only, *after* all the input is read

```
GNU nano 6.2
awk '
BEGIN { print "Analysis of \"caroline\"" }
/caroline/ { ++n }
END {
    print "\"Caroline\" appeared in ", n, "records."
}' employee.txt
```

BEGIN

1. The most common use is to set things like the field separator (FS), output field separator (OFS), or formatting options.

```
awk 'BEGIN {  
    FS = ",";      # Input Field Separator is a comma  
    OFS = "\t"     # Output Field Separator is a tab  
}  
  
    print $1, $2   # Print first and second fields  
' employees.csv
```

Modifiers for **printf** Formats

SYNTAX: %[modifier]control-letter

```
compsys@compsys-virtualbox:~$ awk 'BEGIN{  
> x = 100 * 50  
> printf "The result is %e\n",x  
> }'  
The result is 5.000000e+03
```

e Prints scientific numbers.

BEGIN and END (print headers before any data lines)

```
$awk 'BEGIN {  
    print "Name\tCounty"  
}  
{  
    print $2, $4  
END {  
    print "---- End of Report ----"  
}' employee.txt
```

You can improve alignment using **printf**

```
awk 'BEGIN {  
    printf "%-10s\t%-10s\n",  
    "Name", "Department"  
}  
{  
    printf "%-10s\t%-10s\n", $2, $4  
}  
END {  
    print "---- End of Report ----"  
}' employee.txt
```

BEGIN used to initialise variables

```
$awk 'BEGIN {                                #runs once before any input
    sum = 0; count = 0                        # initialise variables
    }
    {
        sum += $2; count++                    # runs for each matching line
    }
END {
    print "Average:", sum/count
}' grades.txt                                #final calculations
```

Tidy up this layout... what's its output?

```
awk 'BEGIN { printf "%-10s %s\n", "EmpID", "Name" printf "%-10s %s\n", "_____", "_____" } { printf "%-10s %s\n", $1, $2 }'  
employee.txt
```

Common Uses of *awk* in Bash



Text Extraction



```
awk '{ print $1, $3 }' file.txt
```

Extract specific columns

Data Analysis



```
awk '{ sum+=$3 } END { print sum }
```

Calculate sums & averages

Pattern Matching



```
awk '/error/ { print $0 }'
```

Find lines with keywords

Report Formatting



```
awk '{ printf "%-10s %d\n", $1, $2 }'
```

Generate formatted reports

Text Replacement



```
awk '{ gsub("foo", "bar", $2) }'
```

Modify text in fields

Log Processing



```
awk '{ count[$1]++ }  
END { for (ip in count) print ip, count[ip] }'
```

Analyze log files

Try out this weeks Lab